HTTP FOR BACKEND ENGINEERS

Complete Study Notes · Concepts · Definitions · Examples · Diagrams · Interview Revision

---



Client starts communication; server processes and replies.

Learning objective: Understand how browsers, applications, APIs and servers communicate over HTTP, and how backend engineers design reliable, efficient and secure HTTP services.

Prepared as a structured reference for backend engineering and technical interviews.

1. Introduction to HTTP

HTTP (Hypertext Transfer Protocol) is an application-layer communication protocol used to transfer resources and exchange information between clients and servers. Resources can include HTML pages, JSON data, images, videos and files.

OSI layer: Layer 7 – Application Layer. **Default ports:** HTTP uses port 80; HTTPS commonly uses port 443.

Core principles

1. Statelessness: HTTP does not automatically remember previous requests. Each request is processed independently and must carry the information needed for that request, such as a cookie or bearer token when authentication is required. Application sessions can still be implemented using cookies, tokens, databases or shared session stores; statelessness describes HTTP itself.

- **Scalability:** Any suitable server instance can handle a request when required state is available to it.
- **Resilience:** A server restart does not erase state stored by the client or an external session store.
- **Important distinction:** Stateless HTTP does not mean websites cannot remember users. Cookies and server-side session systems provide application-level continuity.

2. Client-server model: The client (browser, mobile app or another service) initiates a request. The server receives it, performs the required work and returns a response.

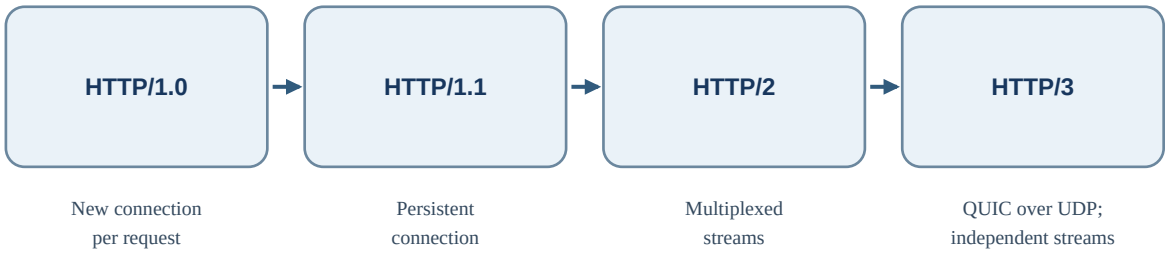


Client starts communication; server processes and replies.

Example: A browser requests `GET /products`. The API server retrieves product data and responds with a status code, headers and usually a response body.

2. Transport Protocols and HTTP Versions

Transport protocol is the layer responsible for moving application data between endpoints. Traditional HTTP/1.1 and HTTP/2 use TCP. HTTP/3 uses QUIC, which runs over UDP and provides reliable, secure streams.



Evolution focuses on connection reuse, concurrency and transport efficiency.

Version	Transport	Key characteristics
HTTP/1.0	TCP	Typically opened a separate connection for each request/response by default; connection reuse was not the default.

Version	Transport	Key characteristics
HTTP/1.1	TCP	Persistent connections by default; connection reuse; Host header; chunked transfer encoding.
HTTP/2	TCP	Binary framing, multiplexing, HPACK header compression; one TCP connection can carry multiple streams.
HTTP/3	QUIC over UDP	Independent streams, integrated TLS 1.3, faster connection setup in many cases; avoids TCP-level head-of-line blocking.

Key definitions

- **Persistent connection / Keep-Alive:** Reuses a connection for multiple exchanges instead of repeatedly creating new connections. In HTTP/1.1, persistence is the default unless the connection is closed.
- **Multiplexing:** Interleaves frames from multiple HTTP/2 or HTTP/3 streams on one connection, allowing concurrent exchanges.
- **Binary framing:** HTTP/2 encodes messages as structured binary frames rather than HTTP/1.1-style textual message syntax.
- **HPACK:** Header compression format used by HTTP/2. HTTP/3 uses QPACK.
- **Head-of-line blocking:** A delay where one blocked item prevents later items from progressing. HTTP/2 can still experience TCP-level head-of-line blocking; QUIC avoids that cross-stream transport blocking.

Interview point: HTTP/3 does not simply send unreliable application data over UDP. QUIC adds reliability, congestion control, encryption and independent streams above UDP.

3. Anatomy of HTTP Messages

An HTTP message is the structured data exchanged between a client and a server. A request travels from client to server; a response travels back. HTTP/1.1 messages are commonly shown in readable text form. HTTP/2 and HTTP/3 use binary framing on the wire.

A. Request message

- **Request line:** Method, target/resource path and HTTP version (in HTTP/1.x).
- **Headers:** Metadata such as Host, Authorization and Content-Type.
- **Blank line:** Separates headers from the message body.
- **Body:** Optional data sent with the request, commonly with POST, PUT or PATCH.

```
POST /api/users HTTP/1.1
Host: example.com
Content-Type: application/json
Authorization: Bearer token123

{ "name": "Sanju", "age": 21 }
```

Part	Meaning
POST	Method: submit data / request an operation.
/api/users	Request target or resource path.
HTTP/1.1	Protocol version in the request line.
Host	Target host name in HTTP/1.1.
Content-Type	Media type of the request body.
Authorization	Credentials or proof of authorization.
JSON body	Application data sent to the server.

B. Response message

- **Status line:** HTTP version, three-digit status code and reason phrase in HTTP/1.x.
- **Headers:** Metadata about the response, such as Content-Type and Cache-Control.
- **Blank line:** Separates headers from the response body.
- **Body:** Optional returned content, such as JSON, HTML or file bytes.

```
HTTP/1.1 201 Created
Content-Type: application/json

{ "id": 101, "name": "Sanju" }
```

Note: The reason phrase (for example, “Created”) is descriptive; clients should rely on the numeric status code.

4. HTTP Headers

HTTP header is a name-value field that carries metadata or instructions associated with a request or response. Headers influence routing, authentication, content interpretation, caching, compression and security.

Header category	Header examples	Purpose
Request	Host, User-Agent, Authorization, Accept, Origin	Describes the client, target, credentials and response preferences.
General / shared	Date, Cache-Control, Connection (HTTP/1.x)	Common message or connection metadata.
Representation	Content-Type, Content-Length, Content-Encoding, Content-Language	Describes the selected representation and its encoding.
Security	Strict-Transport-Security, Content-Security-Policy, X-Content-Type-Options	Enforces or strengthens browser security protections.
Cookie-related	Cookie, Set-Cookie	Sends cookies to a server or asks the browser to store them.

Important header definitions

- **Host:** Identifies the target host in HTTP/1.1 requests.
- **User-Agent:** Identifies the client software making a request.
- **Authorization:** Carries credentials or authorization data, often a Bearer token.
- **Accept:** Lists media types the client can understand, such as application/json.
- **Origin:** Identifies the origin (scheme, host and port) of a browser request.
- **Content-Type:** Identifies the media type of the message body, e.g. application/json.
- **Content-Length:** Gives the message body length in bytes when applicable.
- **Content-Encoding:** Indicates an applied content coding such as gzip or br.
- **Cache-Control:** Specifies caching directives such as max-age, no-cache or no-store.
- **Strict-Transport-Security (HSTS):** Tells browsers to use HTTPS for future requests to the host.
- **Content-Security-Policy (CSP):** Restricts which sources a browser may load or execute, helping mitigate XSS.
- **Set-Cookie:** Instructs the browser to store a cookie.

Cookie security flags

- **HttpOnly:** Prevents client-side JavaScript from reading the cookie through document.cookie; it does not stop the browser sending it with matching requests.
- **Secure:** Restricts cookie transmission to secure channels, normally HTTPS.
- **SameSite:** Controls whether cookies are sent with cross-site requests (Strict, Lax or None). SameSite=None generally requires Secure.

5. HTTP Methods and Idempotency

HTTP method describes the intended action for a request. Methods communicate semantics; the server's route implementation determines the actual business operation.

Method	Definition	Example
GET	Requests a representation of a resource; intended to be read-only.	GET /users/101
POST	Submits data for processing; often creates a resource or triggers an operation.	POST /users
PUT	Creates or replaces the target resource with the supplied representation; commonly used for full replacement.	PUT /users/101
PATCH	Applies a partial modification to a resource.	PATCH /users/101
DELETE	Requests removal of the target resource.	DELETE /users/101
HEAD	Same semantics as GET but asks for response headers without the response body.	HEAD /users/101
OPTIONS	Requests communication options for a resource; used in CORS preflight.	OPTIONS /users

Safe methods

Safe method means the client does not request a change to the server's resource state. GET, HEAD and OPTIONS are defined as safe. Incidental logging or analytics do not make a method unsafe in HTTP semantics.

Idempotency

Idempotency means that making the same request multiple times has the same intended effect on server state as making it once. It does not require identical response bodies or status codes for every repetition.

Method	Idempotent?	Explanation
GET	Yes	Repeated reads do not request a state change.
HEAD	Yes	Repeated metadata requests do not request a state change.
PUT	Yes	Repeatedly setting the same resource representation has the same intended final state.
DELETE	Yes	The resource remains deleted after the first successful deletion; later responses may differ.
OPTIONS	Yes	Repeated capability checks do not request a state change.
POST	Generally no	Repeated submissions can create multiple resources or repeat an operation.
PATCH	Depends	Can be idempotent if the patch operation is designed that way; not guaranteed by the method itself.

Example: Repeating `DELETE /users/101` should leave the user deleted. Repeating `POST /users` may create two users. A PUT that sets a user's name to “Asha” is normally idempotent when repeated with the same body.

6. CORS – Cross-Origin Resource Sharing

Origin is the combination of scheme (protocol), host (domain) and port. For example, https://example.com:443 is an origin. Different schemes, hosts or ports mean different origins.

Same-Origin Policy (SOP) is a browser security rule that restricts scripts from reading resources from a different origin unless the relevant access is permitted.

CORS is a browser-enforced mechanism in which a server uses response headers to tell browsers which cross-origin requests are allowed. It is not a general restriction on server-to-server communication and is not an authentication system.

A. Simple CORS requests

A request is CORS-safelisted only when it meets the defined method, header and Content-Type restrictions. Common examples use GET, HEAD or POST, safelisted headers and one of the permitted form content types: application/x-www-form-urlencoded, multipart/form-data or text/plain.

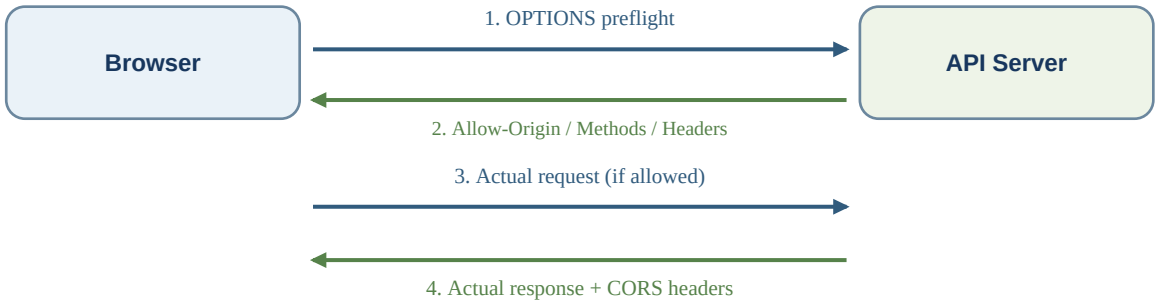
```
Origin: https://frontend.com

Server response:
Access-Control-Allow-Origin: https://frontend.com
```

If the response's CORS headers do not permit the origin, the browser prevents JavaScript from reading the response. The request may still have reached and been processed by the server.

B. Preflight requests

A browser sends a preflight OPTIONS request before certain non-safelisted cross-origin requests. Common triggers include PUT or DELETE, an Authorization header, custom headers or Content-Type: application/json.



Preflight is used only when the browser's CORS rules require it.

```
OPTIONS /api/users HTTP/1.1
Origin: https://frontend.com
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: authorization, content-type
```

Example preflight response:

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://frontend.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Authorization, Content-Type
Access-Control-Max-Age: 3600
```

- **Access-Control-Allow-Origin:** Origins permitted to read the response.
- **Access-Control-Allow-Methods:** Methods permitted for cross-origin requests.
- **Access-Control-Allow-Headers:** Request headers the browser may send.
- **Access-Control-Max-Age:** How long the browser may cache the preflight result.
- **Credentials:** Credentialed CORS requires explicit origin permission and Access-Control-Allow-Credentials: true; wildcard origin (*) cannot be used for credentialed access.

Remember: The browser performs the CORS checks. Tools such as curl or backend services do not enforce browser CORS policy.

7. HTTP Status Codes

Status code is a three-digit number in an HTTP response that indicates the result or current state of the request. The first digit identifies the broad category.

Range	Category	Meaning
1xx	Informational	Request received; process is continuing.
2xx	Successful	Request was successfully received, understood and handled.
3xx	Redirection	Further action, often another request, is needed.
4xx	Client error	The request has a problem or the client is not allowed to perform it.
5xx	Server error	The server failed to fulfil an apparently valid request.

Important status codes

Code	Meaning	Typical use
100 Continue	Client may continue sending request body.	Large request upload handshake.
101 Switching Protocols	Server agrees to switch protocols.	Protocol upgrade.
200 OK	Request succeeded.	Successful GET or general operation.
201 Created	A resource was created.	Successful resource creation.
202 Accepted	Request accepted for processing, not completed yet.	Asynchronous job.
204 No Content	Success with no response body.	Successful update or delete.
301 Moved Permanently	Resource has a permanent new URL.	Permanent redirect.
302 Found	Temporary redirect.	Temporary routing.
303 See Other	Redirect to another resource, typically using GET.	Post/redirect/get flow.
304 Not Modified	Cached representation can be reused.	Conditional GET/HEAD.
307 Temporary Redirect	Temporary redirect preserving method.	Method-preserving redirect.
308 Permanent Redirect	Permanent redirect preserving method.	Method-preserving redirect.
400 Bad Request	Request is malformed or invalid.	Invalid syntax or request data.
401 Unauthorized	Authentication is missing or invalid.	Missing/invalid credentials.
403 Forbidden	Server refuses the action.	Authenticated user lacks permission.
404 Not Found	Target resource not found.	Wrong route or missing resource.
405 Method Not Allowed	Method not supported for this resource.	POST sent to GET-only route.
409 Conflict	Request conflicts with current resource state.	Duplicate username.
415 Unsupported Media Type	Request body media type is unsupported.	Unsupported Content-Type.
422 Unprocessable Content	Content is syntactically valid but cannot be processed.	Validation/business input error.
429 Too Many Requests	Rate limit exceeded.	Too many API calls.
500 Internal Server Error	Unexpected server failure.	Unhandled exception.
501 Not Implemented	Server does not support the required functionality.	Unsupported feature.
502 Bad Gateway	Gateway received an invalid upstream response.	Proxy/load balancer upstream failure.

Code	Meaning	Typical use
503 Service Unavailable	Server temporarily cannot handle the request.	Maintenance or overload.
504 Gateway Timeout	Gateway did not receive an upstream response in time.	Upstream service timeout.

401 vs 403: 401 is about authentication (who are you?); 403 is about authorization (you are not allowed to do this).

8. HTTP Caching

HTTP caching stores reusable response representations so that clients or intermediaries can avoid downloading the same content repeatedly. Caching can reduce latency, bandwidth usage and server load.

Important cache concepts

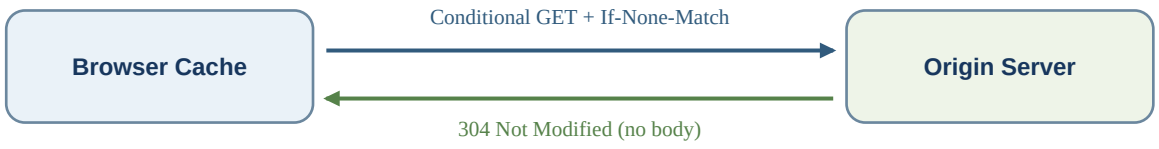
- **Freshness:** A response is fresh while its freshness lifetime has not expired; a fresh response can often be reused without contacting the origin server.
- **Validation:** When a cached response is stale, a cache can ask the server whether its stored representation is still valid.
- **Cache-Control:** Defines directives such as max-age, no-cache, no-store, public and private.
- **ETag:** An entity tag validator identifying a particular representation version.
- **Last-Modified:** Timestamp indicating when the representation was last modified.
- **If-None-Match:** Conditional request header carrying one or more ETags.
- **If-Modified-Since:** Conditional request header carrying a date.

Typical validation flow

```
First response:
HTTP/1.1 200 OK
Cache-Control: max-age=3600
ETag: "abc123"
Last-Modified: Wed, 29 Oct 2026 10:00:00 GMT
```

After the response becomes stale, the browser can revalidate:

```
GET /data.json HTTP/1.1
If-None-Match: "abc123"
```



Browser reuses its stored response body.

If changed, server returns 200 with updated content.

If the representation has not changed, the server can return **304 Not Modified** with no response body; the browser reuses its stored body. If it has changed, the server returns **200 OK** with the updated representation and validator.

no-cache vs no-store: no-cache allows storage but requires revalidation before reuse; no-store instructs caches not to store the response.

9. Content Negotiation and Compression

Content negotiation is the process by which a client and server select an appropriate representation of a resource based on preferences expressed in request headers.

Header	Definition	Example
Accept	Media types the client can process.	application/json
Accept-Language	Preferred natural language(s).	en, hi
Accept-Encoding	Content codings the client supports.	gzip, br
Content-Type	Media type of the representation actually sent.	application/json
Content-Encoding	Content coding applied to the representation.	gzip

```
Client request:
GET /api/data HTTP/1.1
Accept: application/json
Accept-Language: en
Accept-Encoding: gzip, br
```

```
Possible server response:
HTTP/1.1 200 OK
Content-Type: application/json
Content-Encoding: gzip
```

Compression reduces the number of bytes transmitted by encoding content more efficiently. Common content codings include gzip and Brotli (br). The client advertises supported codings with Accept-Encoding; the server identifies the chosen coding with Content-Encoding.

- Less network bandwidth is used.
- Large text responses may download faster.
- Compression effectiveness depends on content; already-compressed images, video and archives may shrink very little.

10. Handling Large Data Transfers

Backend systems often transfer large images, videos, documents, backups and datasets. Efficient transfer avoids unnecessary memory use and supports progress, retries and resumability where needed.

A. Multipart file uploads

multipart/form-data is a media type used to send a form containing multiple parts, such as text fields and binary file data. A boundary string separates the parts in the message body.

```
POST /upload HTTP/1.1
Content-Type: multipart/form-data; boundary=----Boundary123

------Boundary123
Content-Disposition: form-data; name="username"

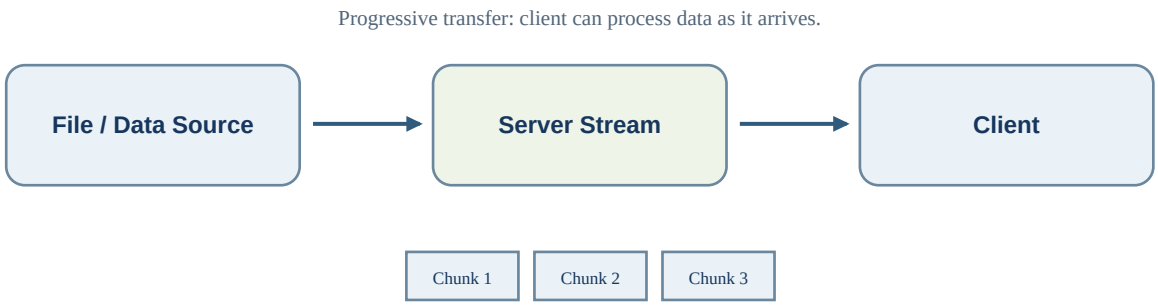
Sanju
------Boundary123
Content-Disposition: form-data; name="file"; filename="photo.png"
Content-Type: image/png

[Binary file data]
------Boundary123--
```

- **Part:** One individual field or file within the multipart body.
- **Boundary:** Delimiter used to identify where each part begins and ends.
- **Content-Disposition:** Describes the form field name and, for files, the filename.
- **Important:** multipart/form-data does not automatically split a file into resumable chunks. Chunked/resumable upload is a separate design, often using upload IDs, chunk numbers and finalization.

B. Streaming downloads

Streaming means sending or processing data progressively rather than first building the entire response in memory. The client can begin receiving data before the complete transfer finishes.



- **HTTP response streaming:** Server writes data progressively to the response.
- **Range requests:** The Range header asks for a portion of a resource; Content-Range describes the returned byte range. Useful for resumable downloads and media seeking.
- **Chunked transfer encoding:** An HTTP/1.1 transfer coding that allows a message body to be sent in chunks when the final size is not known in advance. HTTP/2 and HTTP/3 do not use HTTP/1.1 chunked transfer encoding.
- **Server-Sent Events (SSE):** A one-way server-to-browser event stream using Content-Type: text/event-stream. It is not the general content type for ordinary file downloads.
- **Content-Type: application/octet-stream:** Common generic media type for arbitrary binary data when no more specific type is appropriate.

11. Security – SSL, TLS and HTTPS

SSL (Secure Sockets Layer) is an older protocol family for securing network communication. SSL is obsolete and should not be used.

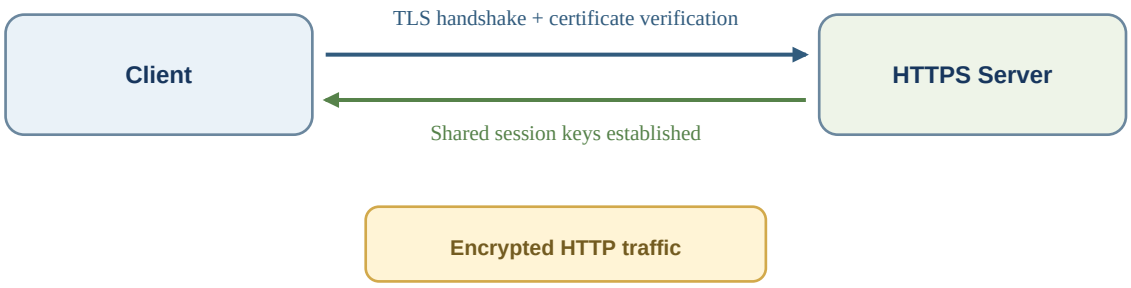
TLS (Transport Layer Security) is the modern cryptographic protocol that protects communication between endpoints. It provides confidentiality, integrity and authentication of the server (and optionally the client).

HTTPS is HTTP carried over a TLS-secured connection. It protects HTTP data while it travels between the client and server.

Three security properties

- **Confidentiality / Encryption:** Prevents unauthorized parties from reading data in transit.
- **Integrity:** Detects unauthorized modification of data in transit.
- **Authentication:** Certificates help the client verify the server's identity. Mutual TLS can also authenticate clients.

Simplified TLS handshake



• Client connects to the server and negotiates TLS parameters.

• Server presents a digital certificate; the client validates its chain, name and validity.

• Both sides establish shared traffic keys using the handshake.

• HTTP messages are then exchanged inside the encrypted TLS connection.

Remember: HTTPS is not a different application protocol from HTTP; it is HTTP protected by TLS. TLS also protects against tampering and impersonation when correctly configured.

12. Quick Revision Sheet

Topic	Key point
HTTP	Application-layer protocol; OSI Layer 7.
Statelessness	Each request is independent; application sessions can still exist.
HTTP/1.1	Persistent connections by default.
HTTP/2	Multiplexing, binary frames, HPACK.
HTTP/3	QUIC over UDP; independent streams.
GET	Read resource; safe and idempotent.
POST	Submit data; generally non-idempotent.
PUT	Replace resource; idempotent.
PATCH	Partial update; idempotency depends on implementation.
DELETE	Delete resource; idempotent.
OPTIONS	Ask about communication options; CORS preflight.
CORS	Browser mechanism for controlled cross-origin access.
304	Cached representation has not changed.
ETag	Representation validator.
Cache-Control	Caching directives.
Accept	Client's preferred media types.
Accept-Encoding	Supported compression codings.
multipart/form-data	Form fields and file upload parts.
SSE	Server-to-browser event stream; text/event-stream.
TLS	Encryption, integrity and authentication.
HTTPS	HTTP over TLS.

Status codes to memorise

1xx	2xx	3xx	4xx	5xx
100 Continue 101 Switching Protocols	200 OK 201 Created 202 Accepted 204 No Content	301 Permanent 302 Found 304 Not Modified 307 Temporary 308 Permanent	400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found 405 Method Not Allowed 409 Conflict 415 Unsupported Media Type 422 Unprocessable Content 429 Too Many Requests	500 Internal Server Error 501 Not Implemented 502 Bad Gateway 503 Service Unavailable 504 Gateway Timeout

13. Backend Interview Questions and Answers

1. What does statelessness mean in HTTP?

Every request is independent at the protocol level. The server does not automatically retain request history; cookies, tokens or external session stores can provide application-level continuity.

2. Which HTTP version introduced multiplexing?

HTTP/2 introduced multiplexing, allowing multiple streams to share one TCP connection. HTTP/3 also supports multiplexed streams using QUIC.

3. Why does HTTP/3 use QUIC over UDP?

QUIC provides reliable, encrypted streams over UDP, supports faster connection establishment and avoids TCP-level head-of-line blocking across independent streams.

4. What is idempotency? Give an example.

Repeated execution has the same intended effect on server state as one execution. PUT and DELETE are defined as idempotent; POST is generally not.

5. What is a CORS preflight request?

A browser-generated OPTIONS request that checks whether a cross-origin request's method and headers are allowed before sending the actual request.

6. Difference between 401 and 403?

401 means authentication is missing or invalid. 403 means the server refuses access, commonly because the authenticated client lacks permission.

7. What is the difference between 304 and 200?

304 tells a cache that its stored representation can be reused and carries no response body. 200 returns a successful representation, normally with a body for GET.

8. What is an ETag?

An ETag is a validator for a selected representation. A client can send it in If-None-Match to revalidate a cached response.

9. Why use multipart/form-data?

It carries multiple form parts, including text fields and binary file data, with boundaries separating each part.

10. Is text/event-stream used for file downloads?

No. text/event-stream is used for Server-Sent Events. Ordinary file downloads use a suitable media type and may use streaming or byte-range responses.

11. What is the difference between no-cache and no-store?

no-cache allows a response to be stored but requires validation before reuse. no-store tells caches not to store the response.

12. What does HTTPS add to HTTP?

TLS adds confidentiality, integrity and server authentication to HTTP communication in transit.

14. Final Checklist

- Can you explain a complete request-response cycle?

- Can you identify request line, headers, blank line and body?
- Can you compare HTTP/1.1, HTTP/2 and HTTP/3?
- Can you explain GET, POST, PUT, PATCH and DELETE?
- Can you distinguish safe and idempotent methods?
- Can you explain CORS simple requests and preflight?
- Can you choose correct status codes for common API outcomes?
- Can you explain Cache-Control, ETag and 304 validation?
- Can you distinguish Accept, Content-Type and Content-Encoding?
- Can you explain multipart upload, streaming and range requests?
- Can you explain TLS, certificates and HTTPS?

Study tip: For backend interviews, practise explaining each concept with one real API example. Focus especially on HTTP methods, status codes, CORS, caching, streaming and TLS.